
```
def __init__(self, window, loc, text,
              width=None,
              height=40,
              textColor=PYGWIDGETS_BLACK,
              upColor=PYGWIDGETS_NORMAL_GRAY,
              overColor=PYGWIDGETS_OVER_GRAY,
              downColor=PYGWIDGETS_DOWN_GRAY,
              fontName=DEFAULT_FONT_NAME,
              fontSize=DEFAULT_FONT_SIZE,
              soundOnClick=None,
              enterToActivate=False,
              callback=None,
              nickname=None):
```

However, rather than reading through the code of a class, a programmer will typically refer to its documentation. As mentioned earlier, you can find the complete documentation for `pygwidgets` at <https://pygwidgets.readthedocs.io/en/latest/>.

You can also view documentation of a class by calling the built-in `help()` function in the Python shell like so:

```
>>> help(pygwidgets.TextButton)
```

When you create an instance of a `TextButton`, you are only required to pass in the window, the location in the window, and the text to be shown on the button. If you only specify these positional parameters, your button will use reasonable defaults for the width and height, the background colors for the four states of the button (different shades of gray), the font, and the font size. By default, no sound effect will be played when the user clicks on the button.

The code to create a `TextButton` using all the defaults looks like this:

```
oButton = pygwidgets.TextButton(window, (50, 50), 'Text Button')
```

The code in the `__init__()` method of the `TextButton` class uses the `pygame` drawing methods to construct its own art for all four states (up, down, over, and disabled). The preceding line creates an “up” version of a button that looks like [Figure 7-2](#).